CoderCommands MCP System

Step 06 Record — FastAPI Gateway

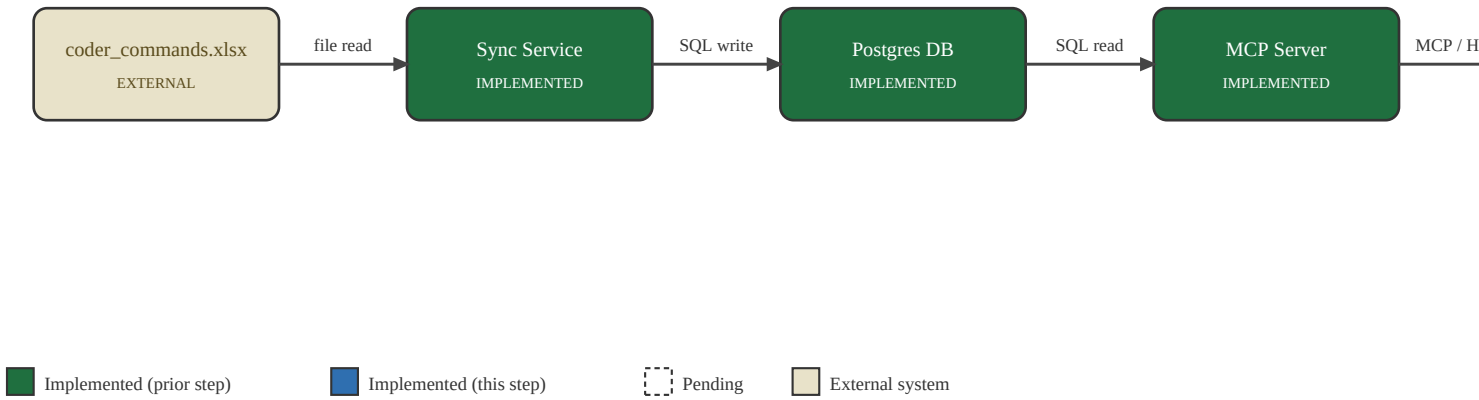
DOCUMENT	CoderCommands MCP System — Step Record
SYSTEM	CoderCommands MCP Command-Reference Server
STEP	06 — FastAPI Gateway
DATE	2026-08-15
REVISION	1.0
STATUS	Complete

1. Purpose & Scope

Give the system one external entry point, separate from the MCP server itself, matching the two-container topology decision. This is the panel-door layer: everything behind it (Postgres, Sync Service, MCP Server) is internal wiring, and this is the only component a client is meant to address directly.

2. System Context Diagram

Signal-flow view of the four functional units (Sync Service, Postgres, MCP Server, FastAPI Gateway) plus the two external endpoints (source workbook, Claude Code client). Fill/border indicates build state as of this step.



3. File / Component Register

Role of every file or component introduced or modified in this step, with its upstream and downstream interfaces (I/O-list style).

Ref	File / Component	Role	Interface In	Interface Out
R1	api/main.py	Byte-level reverse proxy: forwards method, headers (minus hop-by-hop), body, and query string for any request under /mcp to the mcp-server service, and streams the response back unmodified -- required because the MCP Streamable HTTP transport is stateful and uses SSE, not a simple JSON request/response.	Claude Code / any MCP client	mcp-server (Step 5)
R2	api/requirements.txt / Dockerfile	Pins fastapi, uvicorn, httpx; runs uvicorn main:app on port 8100.	—	Docker Compose (Step 7)

4. Work Performed

• Catch-all route /mcp{path:path} forwards GET (SSE stream open), POST (JSON-RPC calls), and DELETE (session termination) to MCP_UPSTREAM_URL, preserving the Mcp-Session-Id header the protocol uses to bind a client to its session on the upstream server.

• Response is relayed via StreamingResponse over the upstream's raw byte stream (httpx aiter_raw), so SSE chunking reaches the client unbuffered rather than being read fully into memory first.

• Added a /healthz endpoint for the Step 7 compose healthcheck.

• httpx.AsyncClient is created and closed via a FastAPI lifespan context rather than a bare module-level instance, so it's torn down cleanly on container shutdown.

• Validated against a real containerized mcp-server (built from the Step 5 Dockerfile) rather than a mock upstream, to exercise the actual MCP protocol semantics through the proxy.

5. Issues Encountered & Resolution

Issue	Root Cause	Resolution
None found in the gateway logic itself on first real end-to-end test.	N/A	Recorded as-is rather than manufacturing a finding: the proxy is a thin, well-scoped component and the design (raw byte streaming, hop-by-hop header stripping, session header passthrough) matched what the protocol needed on the first attempt. One real gap was still found and fixed independent of correctness -- see below.

Issue	Root Cause	Resolution
The httpx.AsyncClient was created as a bare module-level global with no shutdown path.	Not a functional bug at this scale, but a real resource-management gap: the client's underlying connections would never be closed on container stop, which is worth avoiding as a matter of running discipline even in a low-traffic internal tool.	Moved client creation/teardown into a FastAPI lifespan context manager, opened on startup and closed on shutdown.

6. Verification Record

Check	Method	Result
/healthz responds	curl http://127.0.0.1:8100/healthz	**PASS**
mcp-server responds correctly when built and run as an actual Docker container (not just natively via uvicorn)	Direct MCP client call to the containerized server, bypassing the gateway: 16 tools, correct search_commands result	**PASS**
Full MCP session lifecycle proxies correctly: initialize, tool list, tool calls (SSE), session termination	MCP client through the gateway at 127.0.0.1:8100/mcp; cross-checked request/response pairs in both the gateway's and mcp-server's logs -- same session ID created and terminated on both sides	**PASS**
search_commands result through the full proxied path matches the direct-to-container result	Same 'insert a new row into a table' query, compared response bodies	**PASS**
Client teardown behaves correctly after the lifespan refactor	Re-ran the full test suite after the fix; identical results	**PASS**

7. Next Step / Open Dependencies

Step 7 wires all four services together in Docker Compose with proper service names, an internal network, and healthchecks -- replacing the host-published-port test setup used in Steps 4 through 6 with the real deployment topology.